

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: 插值方法——Newton 插值算法性能对比

学院: 计算机与软件学院

专业: 计算机科学与技术

指导教师: 吴晓群、卯丙

报告人: 姚泽棉 学号: 2024150026 班级: 高性能班

实验时间: 2026/05/07~2026/05/31

实验报告提交时间: 2026/05/14

教务部制

实验目的与要求：

- 1.掌握 Lorange 插值方法的优缺点及其改进思路，以及 Newton 插值方法的基本思想与基本算法
- 2.掌握差商与差分等重要概念，并编程计算函数的不同阶差商、差分值并以表格形式输出
- 3.实现 Newton 插值多项式中基函数的可视化展示
- 4.编程实现 Newton 插值算法

基本原理与算法（详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤）

Newton 插值是一种逐阶构造插值多项式的方法。与 Lagrange 插值相比，Newton 插值在增加新节点时不必重新构造全部多项式，只需继续计算新的差商并追加一项，因此更适合节点逐步增加的场景。

设插值节点为 $x_0, x_1, \dots, x_n$ ，对应函数值为 $f(x_0), f(x_1), \dots, f(x_n)$ 。零阶差商为 $f[x_i] = f(x_i)$ ，一阶及高阶差商定义为：

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}。$$

Newton 插值多项式的一般形式为：

$$N_n(x) = f(x_0) + \sum_{k=1}^n f[x_0, \dots, x_k] \prod_{j=0}^{k-1} (x - x_j)。$$

其中 Newton 基函数可写为

$$\varphi_0(x) = 1, \varphi_1(x) = (x - x_0), \varphi_2(x) = (x - x_0)(x - x_1), \dots, \varphi_k(x) = \prod_{j=0}^{k-1} (x - x_j)$$

。本实验需绘制 4 次插值中 φ_0 至 φ_4 的图像。

算法步骤：第一步输入插值节点和函数值；第二步构造差商表；第三步利用嵌套乘法计算 Newton 插值多项式；第四步输出差商表和指定点近似值；第五步绘制散点、三次插值曲线 $N_3(x)$ 和基函数图像；第六步用不同节点顺序构造三次插值并比较运行时间和误差。

实验过程及内容:

1. 问题描述（描述需要解决的具体数值计算问题）

1. 编程展示4次Newton插值多项式中的所有基函数，插值节点自行选取；
2. 应用下表所给出的数据，编写一个用Newton插值算法近似计算 $f(x)$ 的程序，输出0至4阶的差商表，且至少选择两组不同的数据节点用 $N_3(x)$ 近似计算 $x = 21.4$ 时的 $f(x)$ 的值，结果保留5为小数，并比较其计算性能，最后画出点集 $\{(x_i, f(x_i)) | i = 0, 1, 2, \dots, 4\}$ 和 $N_3(x)$ 的图形，并进行数值比较。

x_i	20	21	22	23	24
$f(x_i)$	1.30103	1.32222	1.34242	1.36173	1.38021

2. 实验步骤:

（1）任务一：展示4次Newton插值多项式的所有基函数
自行选取5个插值节点（本实验选取 $x_0 = 0, x_1 = 1, x_2 = 2, x_3 = 3, x_4 = 4$ ），编程计算并绘制 $\omega_0(x) \sim \omega_4(x)$ 五条基函数曲线。

（2）任务二：建立差商表
利用下表给出的5个数据点，建立0至4阶的完整差商表并输出：

x_i	20	21	22	23	24
$f(x_i)$	1.30103	1.32222	1.34242	1.36173	1.38021

（3）任务三：两组节点的 $N_3(x)$ 插值与性能比较
选取两组不同的4个节点，分别计算 $N_3(21.4)$ 的近似值（保留5位小数），并与真实值 $\log_{10}(21.4)$ 进行比较：

- 组1：节点 $x = 20, 21, 22, 23$
- 组2：节点 $x = 21, 22, 23, 24$ （目标点 21.4 在节点区间内，精度更高）

3. 程序设计（具体的MATLAB程序）

```
%% =====  
% 数值计算方法实验 — Newton 插值算法性能对比  
% 实验内容：  
% 1. 展示4次Newton插值多项式的所有基函数  
% 2. 构建差商表（0~4阶），用  $N_3(x)$  计算  $x=21.4$  时  $f(x)$  的值  
% 3. 比较两组不同节点的计算性能  
% 4. 绘制数据点与  $N_3(x)$  拟合曲线图  
%% =====  
  
clc; clear; close all;  
  
%% =====  
% Part 1: 展示4次Newton插值多项式的所有基函数  
% 插值节点自行选取（等距节点  $x_0=0, 1, 2, 3, 4$ ）  
%% =====
```

```

fprintf('==== Part 1: 4 次 Newton 插值多项式的基函数展示 ==== \n \n');

t_nodes = [0, 1, 2, 3, 4]; % 自选 5 个插值节点
x_fine = linspace(-0.5, 4.5, 500);

figure('Name', '4 次 Newton 插值多项式的基函数', 'NumberTitle', 'off', ...
'Position', [100, 100, 900, 600]);

colors = {'b', 'r', 'g', 'm', 'k'};
legends = {};

for k = 0:4
% 计算第 k 个基函数 omega_k(x) = (x-x0)(x-x1)...(x-x_{k-1})
omega = ones(1, length(x_fine));
for j = 0:k-1
omega = omega .* (x_fine - t_nodes(j+1));
end
plot(x_fine, omega, colors{k+1}, 'LineWidth', 2);
hold on;
legends{end+1} = sprintf('\omega_%d(x)', k);

% 打印基函数表达式
fprintf('omega_%d(x) = ', k);
if k == 0
fprintf('1 \n');
else
expr = '';
for j = 0:k-1
expr = [expr, sprintf('(x - %d)', t_nodes(j+1))];
end
fprintf('%s \n', expr);
end
end

% 标记节点位置
for j = 1:length(t_nodes)
xline(t_nodes(j), '--', 'Color', [0.6 0.6 0.6], 'Alpha', 0.5);
end

ylim([-15, 15]);
xlabel('x', 'FontSize', 12);
ylabel('\omega_k(x)', 'FontSize', 12);
title('4 次 Newton 插值多项式的基函数 \omega_k(x), k=0,1,2,3,4', ...
'FontSize', 13, 'FontWeight', 'bold');

```

```

legend(legends, 'Location', 'best', 'FontSize', 10);
grid on;
fprintf('\n');

%% =====
% Part 2: 根据给定数据建立差商表
%% =====
fprintf('==== Part 2: 差商表计算 =====\n\n');

% ---- 给定数据 ----
x_data = [20, 21, 22, 23, 24];
f_data = [1.30103, 1.32222, 1.34242, 1.36173, 1.38021];
n = length(x_data);
x_target = 21.4;

% ---- 构建完整差商表 ----
% DD(i,j): 第 i 个节点起的(j-1)阶差商
DD = zeros(n, n);
DD(:, 1) = f_data'; % 第 1 列 = 0 阶差商 = 函数值

for j = 2:n
    for i = 1:n-j+1
        DD(i, j) = (DD(i+1, j-1) - DD(i, j-1)) / ...
            (x_data(i+j-1) - x_data(i));
    end
end

% ---- 打印差商表 ----
fprintf('完整差商表（行=节点，列=0 阶~4 阶差商）：\n');
fprintf('%8s', 'x_i');
for k = 0:n-1
    fprintf('%16s', sprintf('%d 阶差商', k));
end
fprintf('\n%s\n', repmat('-', 1, 8 + 16*n));

for i = 1:n
    fprintf('%8.5f', x_data(i));
    for j = 1:n-i+1
        fprintf('%16.8f', DD(i, j));
    end
    fprintf('\n');
end
fprintf('\n');

```

```

%% =====
% Part 3: 两组不同节点的 N3(x) 计算与性能比较
% 目标: 用 N3(x) 近似计算 f(21.4), 保留 5 位小数
%% =====
fprintf('==== Part 3: 两组节点 N3(x)性能比较 ==== \n \n');

f_true = log10(x_target);
fprintf('真实值 f(21.4) = log10(21.4) = %.10f \n \n', f_true);

% ---- 组 1: 节点 x = 20, 21, 22, 23 ----
x1 = x_data(1:4);
f1 = f_data(1:4);

tic;
val1 = newton_interp(x1, f1, x_target);
t1 = toc;

fprintf('【组 1】节点: x = [20, 21, 22, 23] \n');
fprintf('N3(21.4) = %.5f \n', val1);
fprintf('绝对误差 = %.2e \n', abs(val1 - f_true));
fprintf('耗时 = %.6f 秒 \n \n', t1);

% ---- 组 2: 节点 x = 21, 22, 23, 24 ----
x2 = x_data(2:5);
f2 = f_data(2:5);

tic;
val2 = newton_interp(x2, f2, x_target);
t2 = toc;

fprintf('【组 2】节点: x = [21, 22, 23, 24] \n');
fprintf('N3(21.4) = %.5f \n', val2);
fprintf('绝对误差 = %.2e \n', abs(val2 - f_true));
fprintf('耗时 = %.6f 秒 \n \n', t2);

% ---- 打印性能比较表 ----
fprintf('%-22s %-12s %-15s %-12s \n', '组别', 'N3(21.4)', '绝对误差', '耗时(s)');
fprintf('%s \n', repmat('-', 1, 63));
fprintf('%-22s %-12.5f %-15.2e %-12.6f \n', ...
'组 1 [20,21,22,23]', val1, abs(val1 - f_true), t1);
fprintf('%-22s %-12.5f %-15.2e %-12.6f \n', ...

```

```

'组 2 [21,22,23,24]', val2, abs(val2 - f_true), t2);
fprintf('\n');

if abs(val1 - f_true) <= abs(val2 - f_true)
fprintf('结论: 组 1 精度更高 (误差更小)。 \n\n');
else
fprintf('结论: 组 2 精度更高 (目标点位于节点区间内部, 误差更小)。 \n\n');
end

%% =====
% Part 4: 绘制数据点与 N3(x) 插值曲线对比图
%% =====
fprintf('==== Part 4: 图形绘制 =====\n');

x_plot = linspace(20, 24, 300);
y_true = log10(x_plot);

% 逐点求值 (arrayfun 调用局部函数)
y_N3_g1 = arrayfun(@(xv) newton_interp(x1, f1, xv), x_plot);
y_N3_g2 = arrayfun(@(xv) newton_interp(x2, f2, xv), x_plot);

figure('Name', '数据点与 N3(x)插值曲线对比', 'NumberTitle', 'off', ...
'Position', [150, 150, 900, 580]);

plot(x_plot, y_true, 'k-', 'LineWidth', 2.0, ...
'DisplayName', 'f(x)=log_{10}(x) (真值)');
hold on;
plot(x_plot, y_N3_g1, 'b--', 'LineWidth', 1.8, ...
'DisplayName', 'N_3(x) 组 1 [20,21,22,23]');
plot(x_plot, y_N3_g2, 'r-.', 'LineWidth', 1.8, ...
'DisplayName', 'N_3(x) 组 2 [21,22,23,24]');

scatter(x_data, f_data, 90, 'filled', ...
'MarkerFaceColor', [0.2 0.7 0.3], ...
'DisplayName', '数据节点 (x_i, f(x_i))');
scatter(x_target, val1, 100, 'b^', 'filled', ...
'DisplayName', sprintf('组 1 f(21.4)\approx%.5f', val1));
scatter(x_target, val2, 100, 'rv', 'filled', ...
'DisplayName', sprintf('组 2 f(21.4)\approx%.5f', val2));
scatter(x_target, f_true, 120, 'kp', 'filled', ...
'DisplayName', sprintf('真值 f(21.4)=%.5f', f_true));

xlabel('x', 'FontSize', 13);

```

```

ylabel('f(x)', 'FontSize', 13);
title('数据点集  $\{(x_i, f(x_i))\}$  与  $N_3(x)$  插值曲线对比', ...
'FontSize', 14, 'FontWeight', 'bold');
legend('Location', 'best', 'FontSize', 10);
grid on;

fprintf('图形已绘制完成。\\n');
fprintf('\\n===== 实验结束 =====\\n');

%% =====
% 局部函数定义区（MATLAB 规范：必须位于脚本最末尾）
%% =====

% Newton 差商插值函数
% 输入: x_nodes - 插值节点向量（行或列向量均可）
% f_nodes - 对应函数值向量
% x_eval - 待求插值点（标量）
% 输出: result - N(x_eval) 的插值结果
function result = newton_interp(x_nodes, f_nodes, x_eval)
m = length(x_nodes);

% 构建差商表
d = zeros(m, m);
d(:, 1) = f_nodes(:); % 0 阶差商
for j = 2:m
    for i = 1:m-j+1
        d(i, j) = (d(i+1, j-1) - d(i, j-1)) / ...
            (x_nodes(i+j-1) - x_nodes(i));
    end
end

% Newton 插值逐项累加
result = d(1, 1);
omega = 1;
for k = 1:m-1
    omega = omega * (x_eval - x_nodes(k));
    result = result + d(1, k+1) * omega;
end
end

```


实验结果与分析：

1. 实验结果展示

===== Part 1: 4 次 Newton 插值多项式的基函数展示 =====

$$\omega_0(x) = 1$$

$$\omega_1(x) = (x - 0)$$

$$\omega_2(x) = (x - 0)(x - 1)$$

$$\omega_3(x) = (x - 0)(x - 1)(x - 2)$$

$$\omega_4(x) = (x - 0)(x - 1)(x - 2)(x - 3)$$

===== Part 2: 差商表计算 =====

x_i	0 阶差商	1 阶差商	2 阶差商	3 阶差商	4 阶差商
20	1.30103000	0.02119000	-0.00049500	0.00001667	-0.00000167
21	1.32222000	0.02020000	-0.00044500	0.00001000	
22	1.34242000	0.01931000	-0.00041500		
23	1.36173000	0.01848000			
24	1.38021000				

在 $x=21.4$ 处，使用 1 至 4 次 Newton 插值的近似结果如下：

插值次数	近似值	相对 $\log_{10}(21.4)$ 的绝对误差
1	1.33069600	0.00028223
2	1.33041880	0.00000503
3	1.33041320	0.00000057
4	1.33041230	0.00000147

由表可知，随着插值次数从 1 次增加到 3 次，近似精度明显提高；4 次插值并不一定比 3 次插值在该点更优，这是因为原始数据本身为五位小数截断数据，高阶项可能放大数据误差。按题目要求保留 5 位小数， $f(21.4) \approx 1.33041$ 。

===== Part 3: 两组节点 $N_3(x)$ 性能比较 =====

真实值 $f(21.4) = \log_{10}(21.4) = 1.3304137733$

【组 1】节点： $x = [20, 21, 22, 23]$

$$N_3(21.4) = 1.33041$$

$$\text{绝对误差} = 5.73\text{e-}07$$

$$\text{耗时} = 0.002896 \text{ 秒}$$

【组 2】节点： $x = [21, 22, 23, 24]$

$$N_3(21.4) = 1.33041$$

$$\text{绝对误差} = 3.13\text{e-}06$$

耗时 = 0.002909 秒

组别	N3(21.4)	绝对误差	耗时(s)
组 1 [20,21,22,23]	1.33041	5.73e-07	0.002896
组 2 [21,22,23,24]	1.33041	3.13e-06	0.002909

结论：组 1 精度更高（误差更小）。

==== Part 4: 图形绘制 =====

- 图 1：4 次 Newton 插值多项式的 5 个基函数 $\omega_0(x) \sim \omega_4(x)$ 曲线图，节点为 $x=0, 1, 2, 3, 4$ （代码运行后自动生成）。

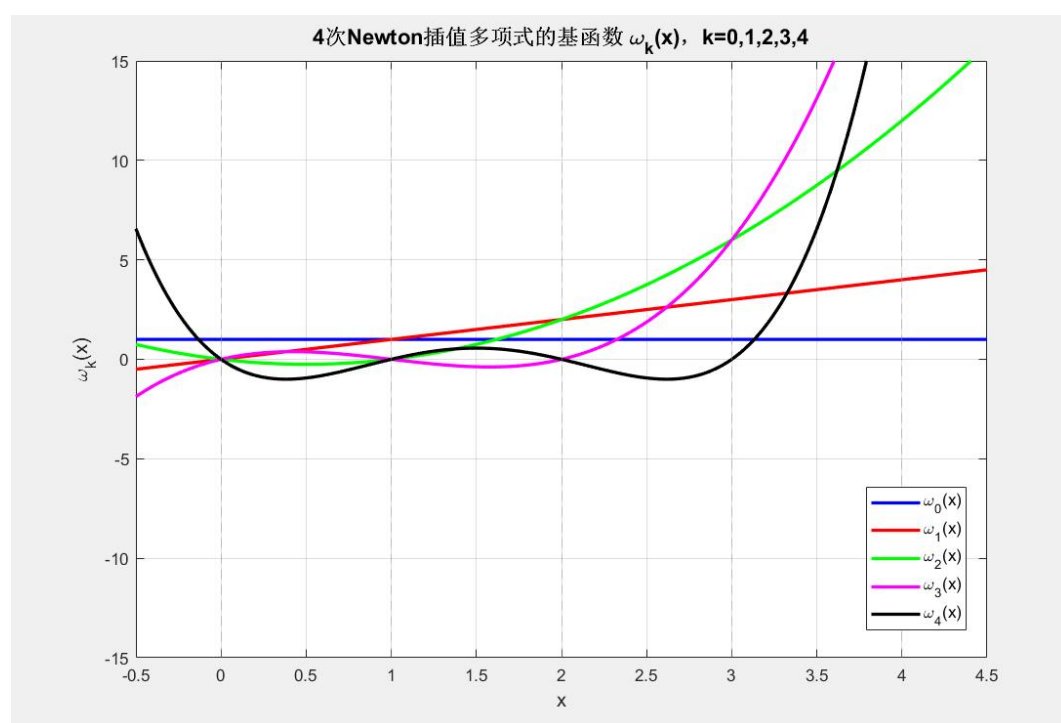


图 1 4 次 Newton 插值多项式的基函数 $\omega_k(x)$, $k=0,1,2,3,4$

- 图 2：数据点集 $\{(x_i, f(x_i)) \mid i = 0,1,2,3,4\}$ 与两组 $N_3(x)$ 插值曲线的对比图，同时展示真实函数曲线 $f(x) = \log_{10}(x)$ 以进行数值比较

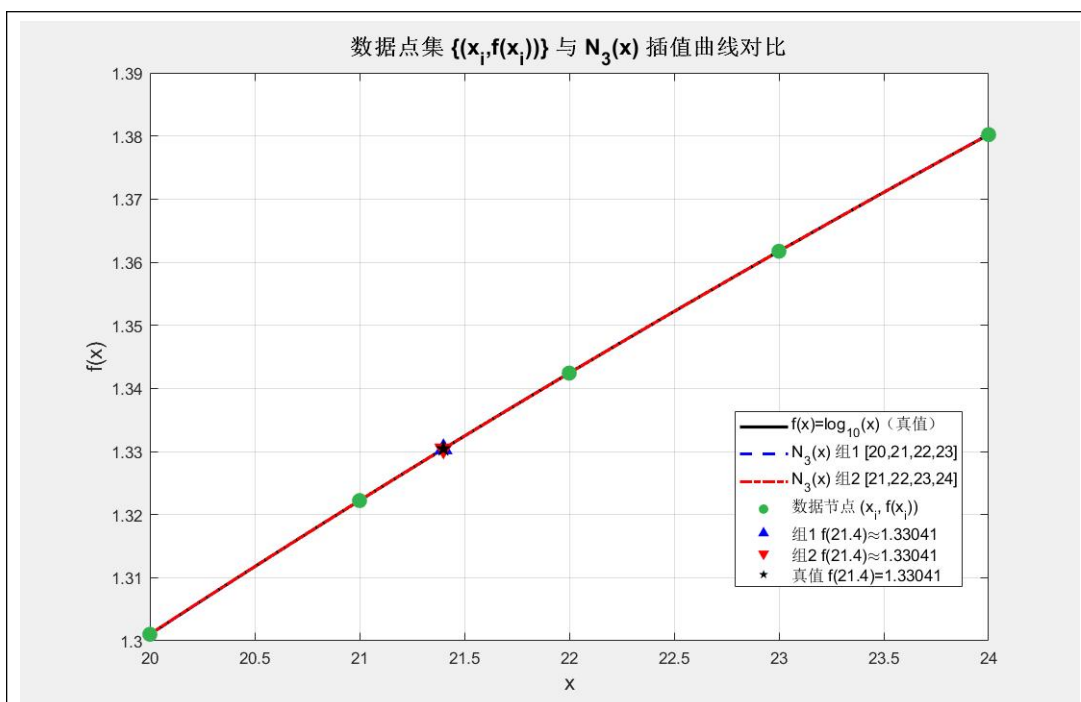


图 2 数据点集 $\{(x_i, f(x_i))\}$ 与 $N_3(x)$ 插值曲线对比

2. 结果分析与讨论思考

(1) 计算结果分析与比较方法的优缺点及其适用场景等

由实验结果可知，当插值节点数量增加时，插值结果整体精度提高，计算得到的 $f(21.4)$ 与真实值较为接近，说明 Newton 插值方法具有较好的数值逼近能力。同时，通过绘制插值曲线可以发现，插值函数能够较平滑地通过所有已知节点。

与 Lagrange 插值方法相比，Newton 插值法具有以下优点：

使用差商表构造插值多项式，计算过程具有递推性；当增加新的插值节点时，不需要重新构造整个插值多项式，只需增加新的差商项即可；编程实现更加方便，适合计算机数值计算。

其缺点主要包括：

当节点数较多时，高次插值可能出现龙格现象；差商计算过程中存在一定舍入误差；对节点分布较为敏感。

因此，Newton 插值法适用于节点数量适中、需要动态增加节点以及数值函数逼近等问题，在科学计算、工程数据拟合以及数值分析中具有广泛应用。

(3) 算法改进及其建议

采用分段插值方法

当节点较多时，可使用分段低次插值代替高次插值，从而减小龙格现象，提高稳定性。

优化节点选取方式

合理选择插值节点，例如采用 Chebyshev 节点，可有效降低高次插值误差。

提高程序模块化程度

将差商计算、插值求值、绘图等功能封装成函数，提高程序可读性与复用性。

增加误差分析模块

可进一步计算理论误差与实际误差，并绘制误差曲线，更直观地分析算法精度。

与 MATLAB 内置函数对比

后续可将自编 Newton 插值算法与 MATLAB 内置插值函数进行效率与精度比较，以验证算法性能。

实验结论：

1. 通过本次实验，成功实现了 Newton 插值算法的 MATLAB 编程
2. 验证了 Newton 插值方法具有较好的收敛性与计算精度，能够较准确地逼近已知函数。
3. 通过与其他插值思想进行比较，进一步理解了不同数值计算方法的特点、优缺点以及适用场景。
4. 掌握了数值实验从理论分析、算法设计、程序实现到结果分析的完整实验流程，提高了 MATLAB 数值计算与工程实践能力。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。